HTTP 완벽 가이드 · Week 6

캐시 (1)

동작 원리와 신선도 · 7장 · p.185 ~ 209

정우인

부록 · 들어가며

‘캐시’는 어디에나 있다 — 오늘은 HTTP 캐시

컴퓨터 구조 · OS 에서 말하는 캐시

- CPU 캐시 메모리 (L1/L2/L3)
- MESI 프로토콜
- 페이지 캐시
- ···

백엔드 개발자가 주로 신경 쓰는 캐시는 두 가지

1 애플리케이션 캐시

무엇을	DB 조회 결과 · 계산 결과 같은 데이터
어디에	서버 메모리 (로컬) / Redis (분산)
누가 제어	애플리케이션 코드

2 HTTP 캐시

이번 주 주제

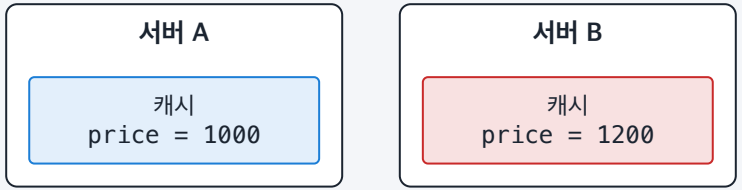
무엇을	HTTP 응답 메시지의 사본
어디에	브라우저 · 프록시 · CDN
누가 제어	HTTP 헤더 (Cache-Control 등)

→ 이 장의 캐시 = **HTTP 응답의 사본**을 어딘가에 두고 다시 쓰는 것

애플리케이션 캐시 — 로컬 vs 분산

1 로컬 캐시

서버 메모리 안에 저장 (Caffeine 등)

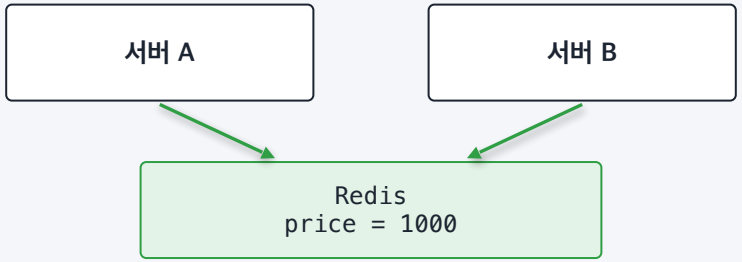


≠ 같은 요청인데 서버마다 다른 값

+ 네트워크 없이 메모리 접근 → 가장 빠름
– 서버가 여러 대면 내용이 달라질 수 있음

2 분산 캐시

별도 캐시 서버에 저장 (Redis 등)

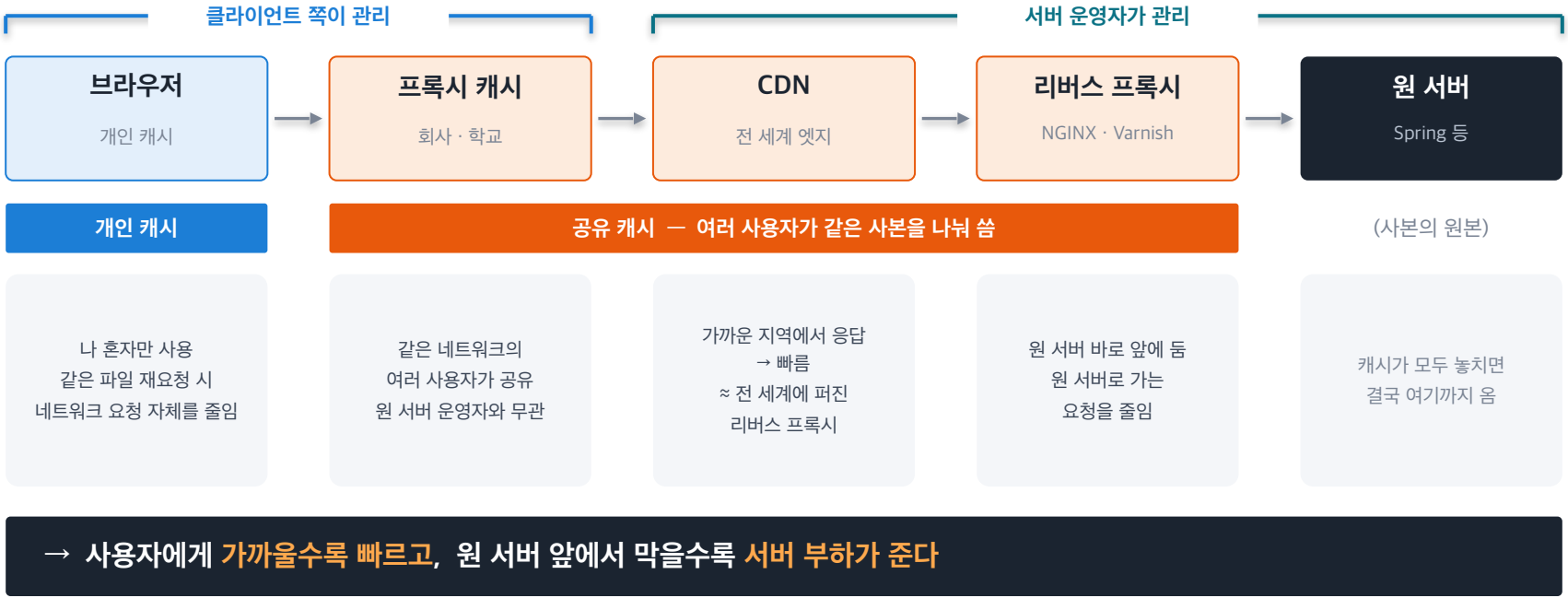


+ 모든 서버가 같은 캐시를 공유
– 네트워크를 한 번 거침 (로컬보다 느림)

→ 둘 다 서버 안쪽에서 데이터를 아끼는 캐시 — HTTP 캐시는 응답이 오가는 길목에 둔다

들어가며

HTTP 캐시 — 응답이 지나가는 길목마다

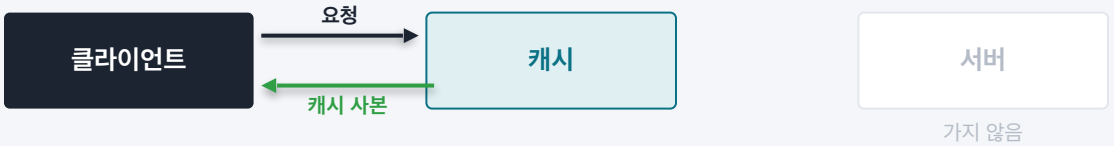


→ 사용자에게 가까울수록 빠르고, 원 서버 앞에서 막을수록 서버 부하가 준다

사본이 있으면 적중, 없으면 부적중

1 캐시 적중

요청에 대응하는 사본이 있음
→ 그 사본으로 바로 응답



2 캐시 부적중

대응하는 사본이 없음
→ 원 서버로 그대로 전달



→ 캐시가 **혼자 처리**하면 적중, **서버까지 가야** 하면 부적중

사본이 오래됐다면 — 서버에 물어본다

재검사 = 신선도 검사

갖고 있는 사본이 여전히 최신인지 서버를 통해 때때로 점검
대부분은 사본이 **검사할 만큼 충분히 오래된 경우에만** 재검사

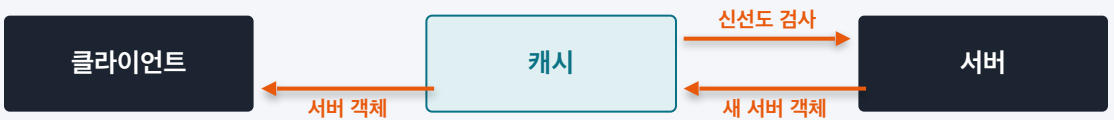
a 재검사 적중

"느린 적중"
서버 객체 = 캐시된 사본



b 재검사 부적중

사본의 유효기간이 지남
서버 객체 ≠ 캐시된 사본



→ 서버엔 갔지만 "그대로 써도 돼"면 재검사 적중, 새 객체를 받아오면 재검사 부적중

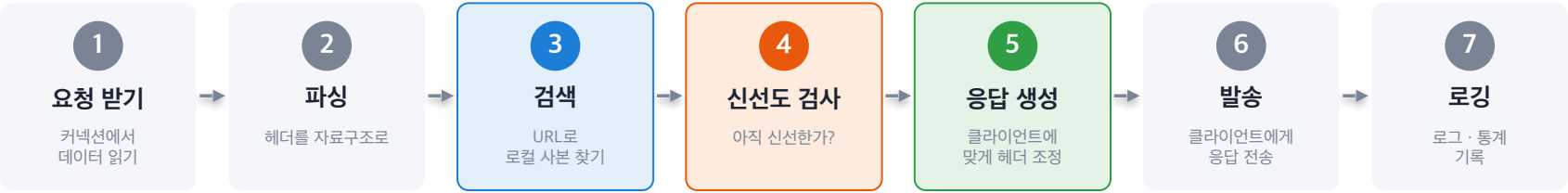
네 가지 경우 — 무엇이 얼마나 빠른가

느림 → 빠름

재검사 부적중	=	캐시 부적중	재검사 적중	캐시 적중
사본: 있음 · 바뀜		사본: 없음	사본: 있음 · 오래됨	사본: 있음 · 신선
서버 왕복: 새 객체 받아옴		서버 왕복: 원본 받아옴	서버 왕복: 확인만 다녀옴	서버 왕복: 안 감
본문 출처: 서버		본문 출처: 서버	본문 출처: 캐시	본문 출처: 캐시

→ 재검사 부적중 = 캐시 부적중 < 재검사 적중 < 캐시 적중

요청이 오면 캐시는 일곱 단계를 거친다



3 검색

URL로 로컬 사본이 있는지 확인
없으면 → 원 서버나 부모 프록시에서
가져옴 (또는 실패)

4 신선도 검사

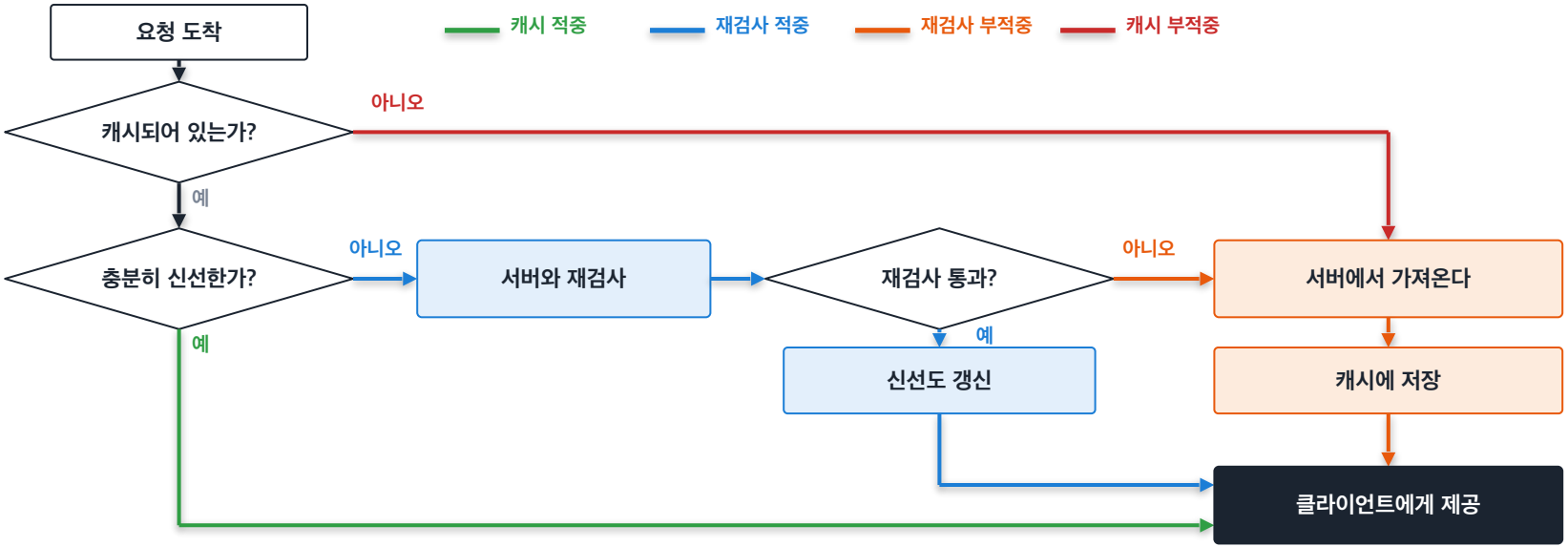
보유 기간 안 → 신선, 서버 접촉 없이 제공
신선도 한계를 넘김 → 신선하지 않음
→ 제공 전에 서버와 재검사

5 응답 생성

클라이언트에 맞게 헤더 조정
(버전 · 신선도 정보 · Via 헤더 등)
단, **Date 헤더는 건드리지 않음**

→ 핵심은 가운데 셋 — 찾고 → 신선한지 보고 → 헤더 맞춰 응답

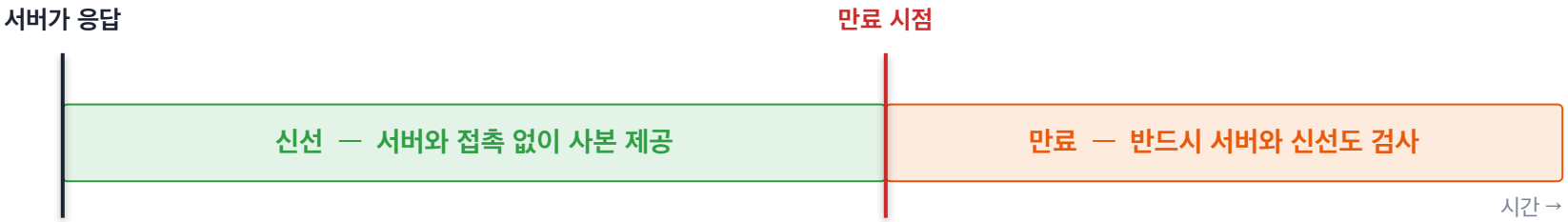
GET 요청 한 건이 지나가는 길



→ 앞에서 본 네 가지 경우가 곧 이 플로 차트의 네 갈래 길

7.8.1 문서 만료

만료 전엔 그냥 쓰고, 만료 후엔 꼭 확인한다



만료 시점은 누가 알려주나? → 서버가 응답 헤더로

1 Cache-Control

상대 시간 — 몇 초 동안

```
Cache-Control: max-age=484200
```

2 Expires

절대 시간 — 이 시각까지

```
Expires: Fri, 05 Jul 2002, 05:00:00 GMT
```

→ 캐시된 데이터는 서버 데이터와 일치하도록 — 그 기준이 만료 시점

max-age는 '몇 초 동안', Expires는 '언제까지'

1 Cache-Control: max-age

상대 시간

```
Cache-Control: max-age=484200
```

"만들어진 지 몇 초까지 신선하다"

484200초 ≈ 5일 14시간 30분 동안 서버에 안 묻고 사용

\+ 기준이 '받은 시점부터 몇 초' → 시계가 안 맞아도 OK

2 Expires

절대 시간

```
Expires: Fri, 05 Jul 2002, 05:00:00 GMT
```

"이 날짜 · 시각까지 신선하다"

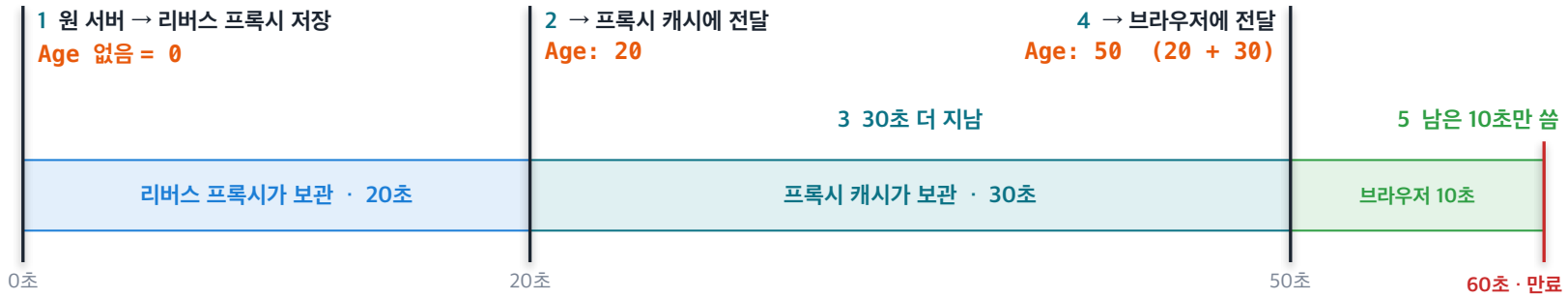
그 시각이 지나는 순간부터 신선하지 않음

– 서버 · 캐시 시계가 어긋나면 만료 판단이 틀어짐

→ 둘 다 있으면 Cache-Control: max-age 가 우선

캐시를 몇 번 거쳐도, 나이는 이어서 센다

예시 `Cache-Control: max-age=60`



Age += 내가 들고 있던 시간

캐시가 몇 단계든, 원 서버가 응답한 뒤
실제로 지난 시간이 그대로 유지

남은 신선 시간

max-age − Age = 60 − 50 = 10초

→ max-age는 '총 수명', Age는 '이미 산 나이' — 남은 건 그 차이

재검사는 ‘조건부 GET’ 한 번으로

조건부 GET = GET 요청 + **조건부 헤더** → 조건이 참일 때만 객체를 보냄



→ 신선도 검사와 객체 받아오기를 요청 한 번에 — 그래서 조건부 GET = 재검사

조건부 헤더는 5가지
— 오늘은 두 가지만

If-Modified-Since
날짜로 비교

If-None-Match
ETag(버전 이름표)로 비교

→ 서버는 조건이 참일 때만 본문을 보낸다 — 아니면 304로 ‘그대로 써’

“이 날짜 이후 바뀌었으면 보내 줘”

1 처음 받을 때 (서버 → 캐시)

```
Last-Modified: Sat, 29 Jun 2002, 14:30:00 GMT
```

같은 날짜 돌려보냄

2 재검사할 때 (캐시 → 서버)

```
If-Modified-Since: Sat, 29 Jun 2002, 14:30:00 GMT
```

그 뒤에 변경됨 → 조건 참

```
HTTP/1.0 200 OK
Last-Modified: <새 날짜>
Expires: <새 만료 날짜>

<새 문서 본문>
```

새 문서 + 새 만료 날짜 + 그 외 정보

변경 없음 → 조건 거짓

```
HTTP/1.0 304 Not Modified
Expires: <새 만료 날짜>
Cache-Control: …

(본문 없음)
```

갱신이 필요한 헤더만 담아 보냄

→ 안 바뀌었으면 304 Not Modified — 본문 없이 갱신할 헤더만

날짜만으로는 부족한 경우들

1 내용은 그대로, 시각만 바뀜

일정 간격으로 다시 쓰이는 문서
데이터는 같은데 변경 시각은 새로

2 다시 받기엔 사소한 변경

철자 · 주석 수정 정도인데
전 세계 캐시가 다시 읽어야 함

3 변경 일시를 정확히 모름

어떤 서버는 페이지의 최근 변경
일시를 정확히 판별하지 못함

4 1초보다 짧은 갱신

날짜는 초 단위
→ 1초 안의 여러 번 변경은 구분 불가

→ 날짜 대신 내용 버전 이름표로 비교하자 = 엔터티 태그 (ETag)

ETag가 같으면 304, 다르면 200 + 새 ETag

캐시 → 서버

```
GET /announce.html HTTP/1.0
If-None-Match: "v2.6"
```

"내가 가진 건 v2.6 —
이게 아니면 보내 줘"

서버 ETag도 v2.6
같음

```
HTTP/1.0 304 Not Modified
ETag: "v2.6"
```

서버 ETag는 v3.0
바뀜

```
HTTP/1.0 200 OK
ETag: "v3.0"    + 새 콘텐츠
```

태그를 여러 개 보낼 수도 있다

```
If-None-Match: "v2.4", "v2.5", "v2.6"
```

서버 태그가 이 중 하나와 같으면 304

둘 다 보내면?

If-None-Match 가 우선
If-Modified-Since를 같이 두는 이유 = 호환성
(ETag를 처리 못 하는 오래된 캐시 · 프록시)

→ 날짜가 아니라 **내용 버전**을 비교 — 바뀌었을 때만 **새 본문 + 새 ETag**

검사기 — 강한 것과 약한 것

검사기(validator) = 내 사본이 최신인지 확인할 때 쓰는 값 — `ETag` `Last-Modified`

1 강한 검사기

내용이 **1바이트라도** 바뀌면 값이 바뀜

→ 바이트 단위로 완전히 같아야 '같음'

예

```
ETag: "v2.6"
```

2 약한 검사기

사소한 변경은 같은 걸로 쳐 줌

의미 있는 변경일 때만 값이 바뀜

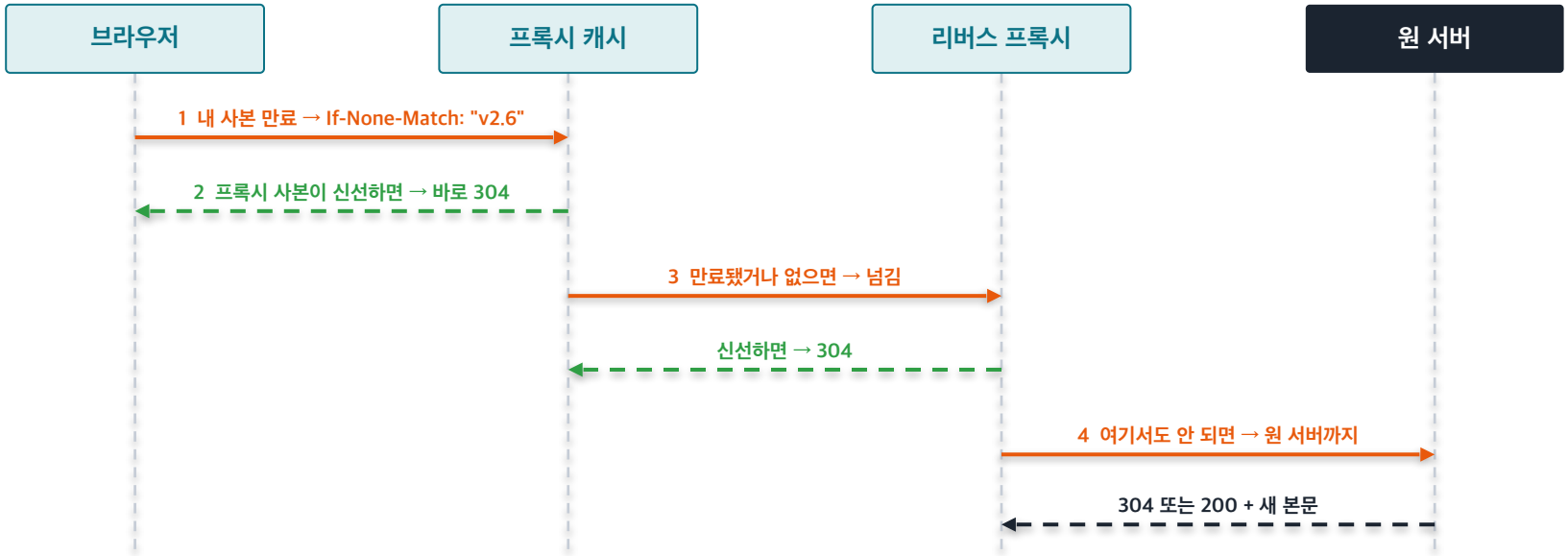
`Last-Modified`도 약한 검사기 취급

W/ 를 붙여서 구분

```
ETag: W/"v2.6"
```

→ 주의: **ETag는 재활용하지 말 것** — 다른 내용에 옛 태그를 쓰면 캐시가 같은 걸로 착각

캐시가 여러 개라면? — 가까운 곳부터 차례로



→ 각 캐시의 '신선한가?'는 max-age − Age로 판단 — Age 예시는 12쪽